

Java desde cero

Presentaciones para curso de 40 horas

10 módulos · teoría + práctica + proyecto

Java 21 / 25 LTS

```
public class Main {  
    public static void main(String[]  
args) {  
        System.out.println("Hola,  
Java");  
    }  
}
```

Objetivo: que el alumnado escriba, entienda, depure y estructure programas simples.

Mapa del curso: 40 horas

1 Entorno y sintaxis

**2 Variables y
operadores**

3 Condicionales

4 Bucles

5 Métodos

**6 Arrays y
colecciones**

7 POO

**8 Herencia e
interfaces**

**9 Excepciones y
ficheros**

10 Proyecto final

Estructura recomendada: 35% explicación, 55% práctica guiada, 10% revisión de errores.

Módulo 1 · Entorno, JDK e IDE

Instalar JDK y configurar PATH.
Elegir IDE: IntelliJ IDEA, Eclipse o VS Code.
Diferenciar JDK, JRE, JVM y bytecode.
Compilar y ejecutar desde terminal.



```
javac Main.java  
java Main
```

Ejercicio: instalar el entorno y ejecutar un “Hola mundo” desde el IDE y desde la terminal.

Módulo 2 · Primer programa y estructura básica

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hola Yamilet");  
    }  
}
```

La clase pública debe coincidir con el nombre del archivo.

main es el punto de entrada del programa.

Las llaves delimitan bloques.

Cada instrucción termina con punto y coma.

Práctica: cambiar el mensaje, provocar un error de compilación y leer el aviso del IDE.

Módulo 3 · Variables y tipos de datos

int	enteros	42
double	decimales	19.95
boolean	verdadero/falso	true
char	carácter	'A'
String	texto	"Java"

```
int edad = 43;  
double precio = 12.99;  
boolean activo = true;  
String nombre = "Coke";
```

Ejercicio 1: ficha personal por consola.

Ejercicio 2: calcular precio con IVA.

Ejercicio 3: convertir Celsius a Fahrenheit.

Módulo 4 · Operadores y entrada por teclado

Aritméticos: +, -, *, /, %.

Comparación: ==, !=, >, <, >=, <=.

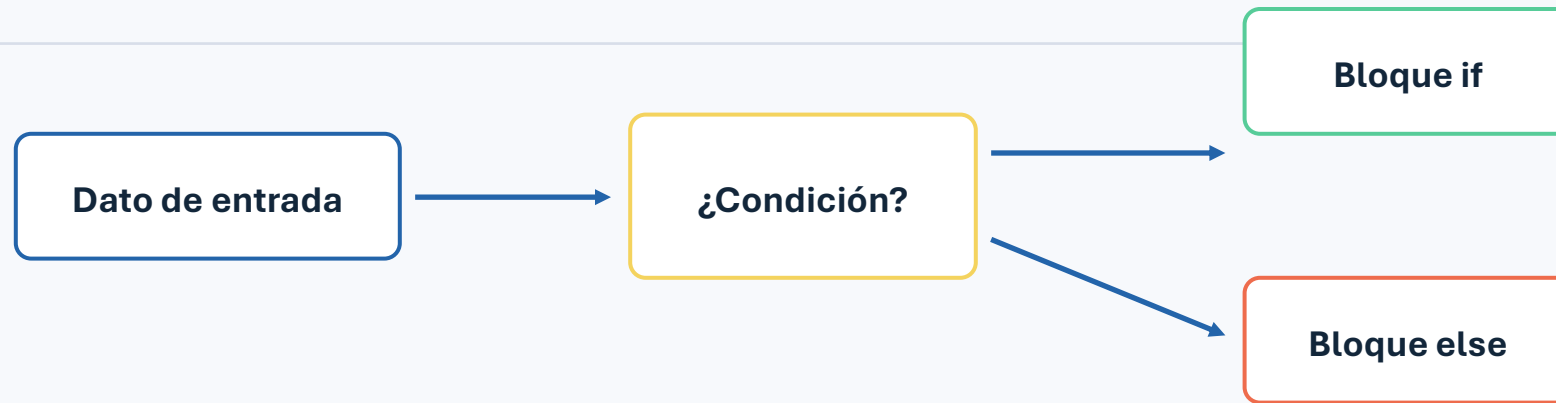
Lógicos: &&, ||, !.

Scanner permite leer datos introducidos por el usuario.

```
Scanner sc = new Scanner(System.in);  
System.out.print("Edad: ");  
int edad = sc.nextInt();  
System.out.println(edad >= 18);
```

Error frecuente: comparar texto con == en vez de usar .equals().

Módulo 5 · Condicionales



```
if (nota >= 5) {  
    System.out.println("Aprobado");  
} else {  
    System.out.println("Suspenso");  
}
```

Ejercicio: clasificar nota en insuficiente, aprobado, notable o sobresaliente.

Ejercicio: menú simple con switch.

Módulo 6 • Bucles

```
for (int i = 1; i <= 10; i++) {  
    System.out.println(i);  
}  
  
while (opcion != 0) {  
    // repetir menú  
}
```

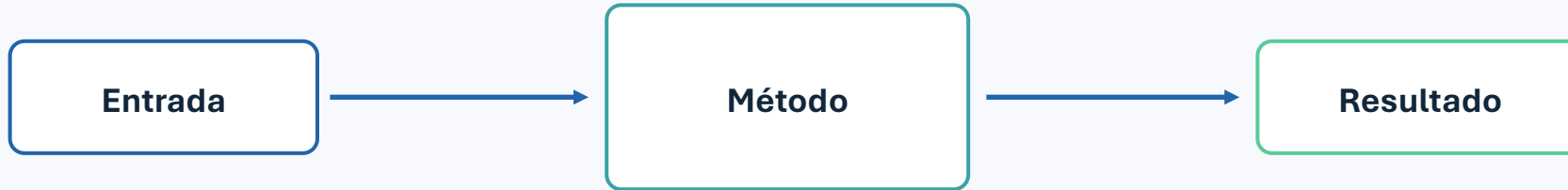
for: cuando sabemos cuántas repeticiones habrá.

while: cuando dependemos de una condición.
do while: ejecuta al menos una vez.

Evitar bucles infinitos: actualizar la variable de control.

Práctica: tabla de multiplicar, suma acumulada y menú repetitivo.

Módulo 7 • Métodos



```
static double calcularIVA(double precio) {  
    return precio * 1.21;  
}
```

Reduce duplicación.
Hace el código más legible.
Permite probar partes pequeñas del programa.
Parámetros = datos que entran; return = dato que sale.

Módulo 8 · Arrays y colecciones

```
int[] notas = {7, 5, 9};
ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");
HashMap<String, Integer> stock = new HashMap<>();
stock.put("Velas", 20);
```

Array: tamaño fijo.

ArrayList: lista dinámica.

HashMap: pares clave-valor.

foreach: recorrer sin manejar índices manualmente.

Ejercicio: lista de productos, cálculo de total y búsqueda por nombre.

Módulo 9 · Programación orientada a objetos



```
class Producto {  
    String nombre;  
    double precio;  
  
    void mostrar() {  
        System.out.println(nombre + " " + precio);  
    }  
}
```

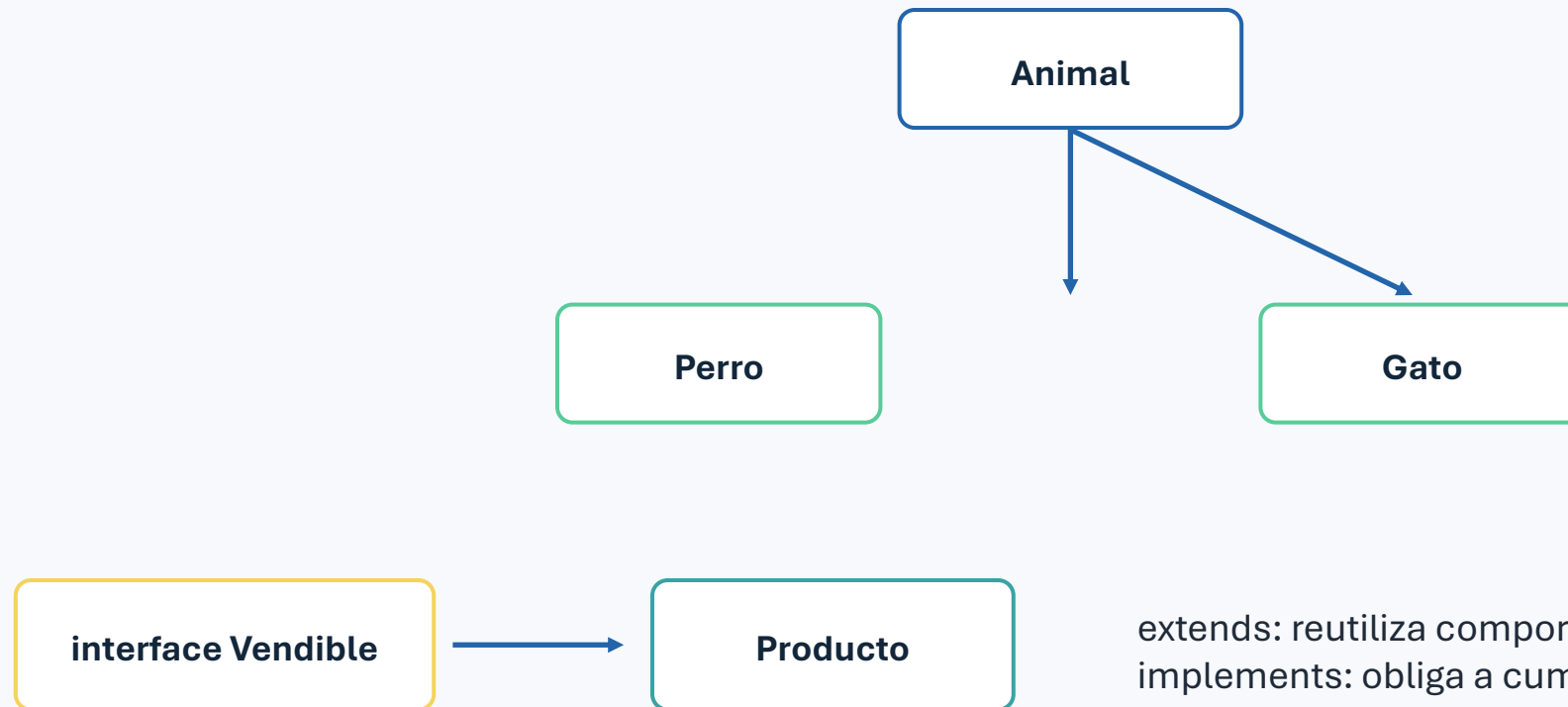
Atributos: datos del objeto.
Métodos: acciones del objeto.
new crea una instancia.

Módulo 10 · Encapsulación y constructores

```
class Producto {  
    private String nombre;  
    private double precio;  
  
    public Producto(String nombre, double precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    public double getPrecio() { return precio; }  
}
```

private protege los datos.
Constructor inicializa el objeto.
getters/setters controlan acceso y validación.
this diferencia atributo y parámetro.

Módulo 11 · Herencia e interfaces



extends: reutiliza comportamiento de una clase base.

implements: obliga a cumplir un contrato.

No abusar de herencia; preferir composición cuando sea más claro.

Módulo 12 · Excepciones

```
try {  
    int numero = Integer.parseInt(texto);  
} catch (NumberFormatException e) {  
    System.out.println("Número no válido");  
} finally {  
    System.out.println("Fin");  
}
```

Una excepción es un fallo controlable en tiempo de ejecución.

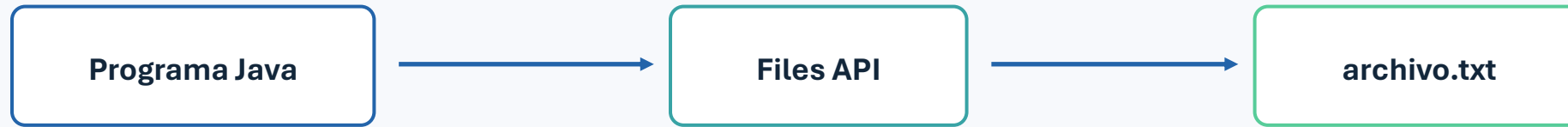
try contiene código arriesgado.

catch define qué hacer ante el error.

finally se ejecuta siempre.

Práctica: validar edad, precio y opción de menú sin que el programa se cierre.

Módulo 13 · Ficheros



```
Path ruta = Path.of("datos.txt");  
Files.writeString(ruta, "Hola Java");  
String texto = Files.readString(ruta);
```

Guardar datos simples.
Leer configuraciones o listados.
Tratar IOException.

Módulo 14 · Depuración y errores típicos

Error de compilación: el código no se traduce a bytecode.

Error de ejecución: compila, pero falla al ejecutar.

Error lógico: funciona, pero calcula mal.

Usar breakpoints, step over y watch variables.

```
// Error lógico  
int total = precio * cantidad;  
// ¿precio debería ser double?  
// ¿cantidad puede ser 0?
```

Regla docente: el alumno debe explicar el error antes de copiar la solución.

Banco de ejercicios por bloques

Básico

Hola mundo
Datos personales
IVA
Conversor

Control

Mayor de edad
Notas
Menú
Tabla multiplicar

Estructura

Métodos
Arrays
ArrayList
HashMap

POO

Producto
Cliente
Factura
Inventario

Secuencia: explicar → copiar ejemplo → modificar ejemplo → resolver desde cero.

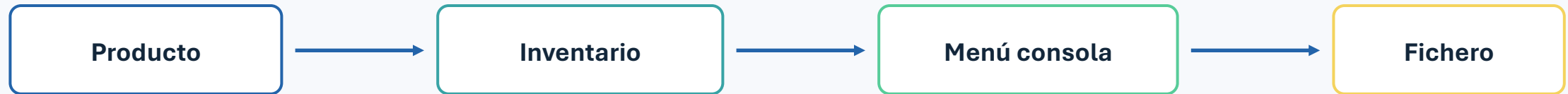
Prueba práctica intermedia

Crear una calculadora de consola.
Pedir dos números al usuario.
Mostrar menú: sumar, restar, multiplicar, dividir, salir.
Validar división por cero.
Repetir hasta elegir salir.

1. Sumar
2. Restar
3. Multiplicar
4. Dividir
0. Salir

Duración sugerida: 60–75 min

Proyecto final · Gestión de inventario



Alta de producto: nombre, precio y stock.
Listado completo.
Búsqueda por nombre.
Entrada/salida de unidades.
Persistencia en fichero.

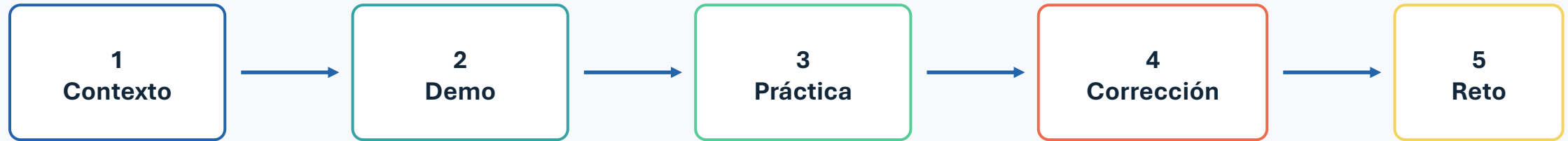
Requisito mínimo: clases y métodos.
Extra: excepciones y validación robusta.
Extra: exportar informe de bajo stock.

Rúbrica de evaluación

Criterio	Peso
Funciona y compila	30%
Estructura y legibilidad	25%
Uso correcto de POO	20%
Validación de errores	15%
Autonomía y explicación	10%

Evaluar código que se entiende, no solo código que “parece” funcionar.

Guía para impartir cada sesión



No explicar 40 minutos seguidos: Java se aprende escribiendo código.
Forzar lectura de errores: es una habilidad central.
Corregir nombres, indentación y duplicación desde el día 1.

Cierre · Lo que debe saber hacer el alumno

Crear un proyecto Java desde cero.
Usar variables, condicionales, bucles y métodos.
Manipular listas, mapas y arrays.
Diseñar clases simples con atributos y métodos.
Leer, escribir y validar datos.
Explicar errores frecuentes y depurar.

```
// Indicador real de aprendizaje  
No es memorizar sintaxis.  
Es resolver un problema pequeño  
y explicar por qué funciona.
```

Siguiente paso natural: Java + bases de datos + Spring Boot básico.